Crane: Inter-Layer Scheduling Framework for DNN Inference and Training Co-Support on Tiled Architecture

Original Paper by: Yu Gong et al. (MICRO ‘2025)

Presented by Jongyun Hur

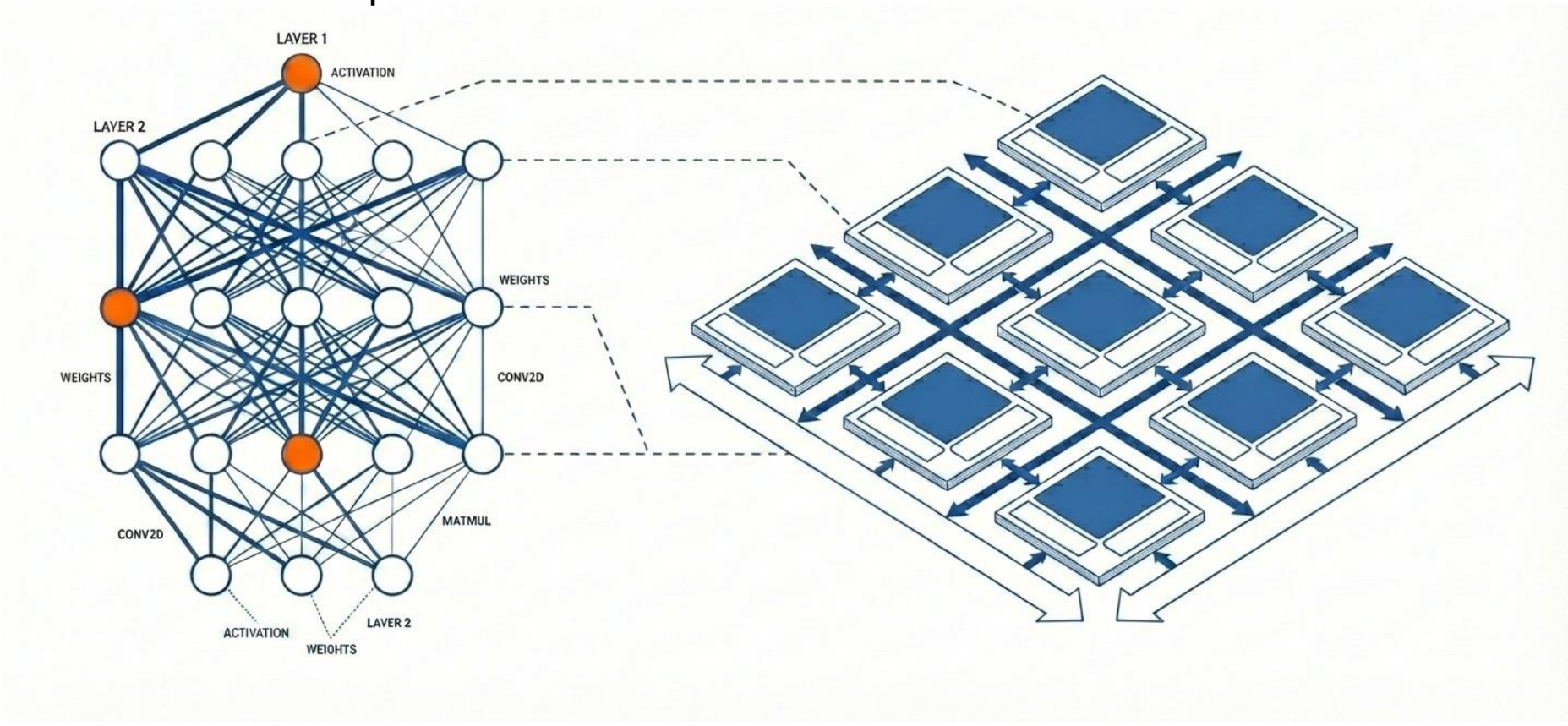
Introduction

Introduction

Tiled Architecture: Hardware for DNN scalability and efficiency

DNN Graph

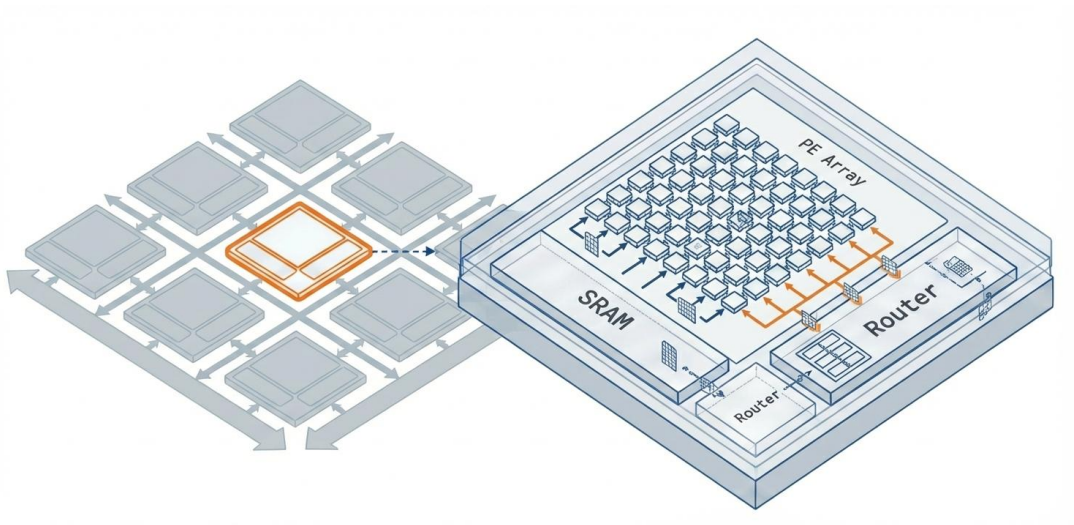
Tiled Architecture



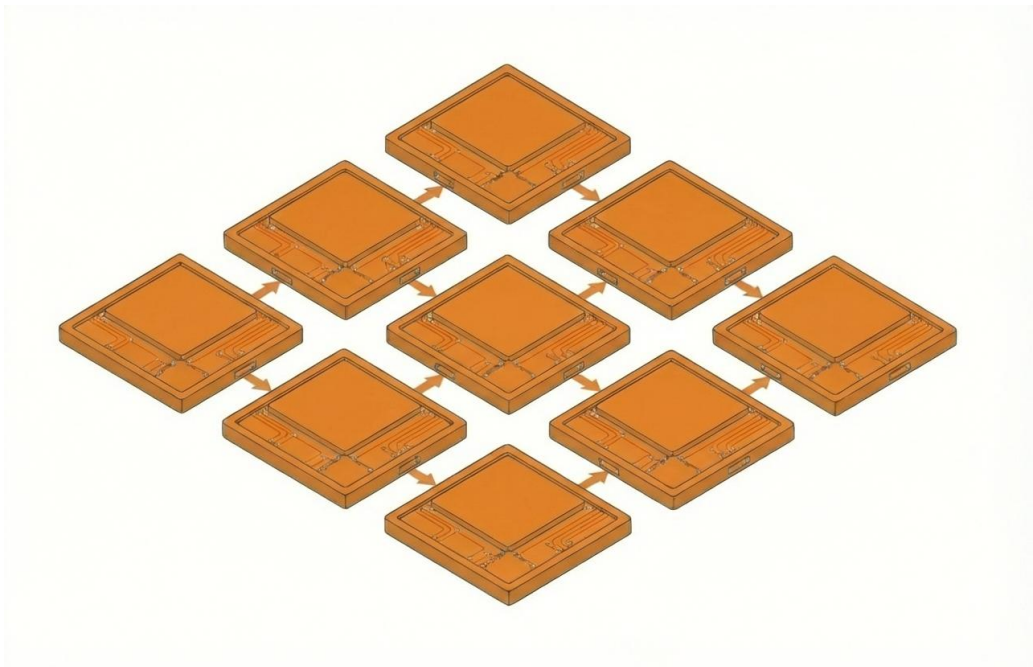
Introduction

Intra-layer Scheduling vs. Inter-layer Scheduling

Intra-layer Scheduling

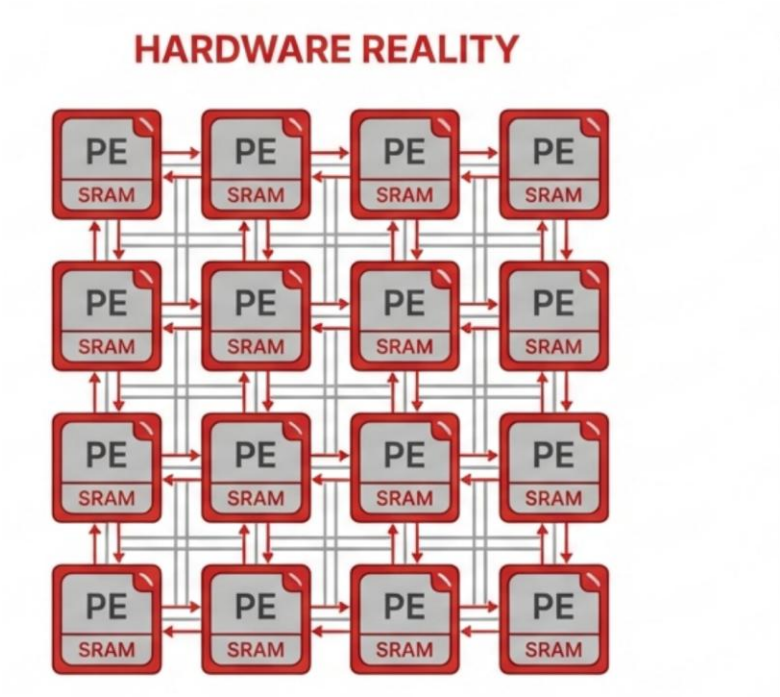


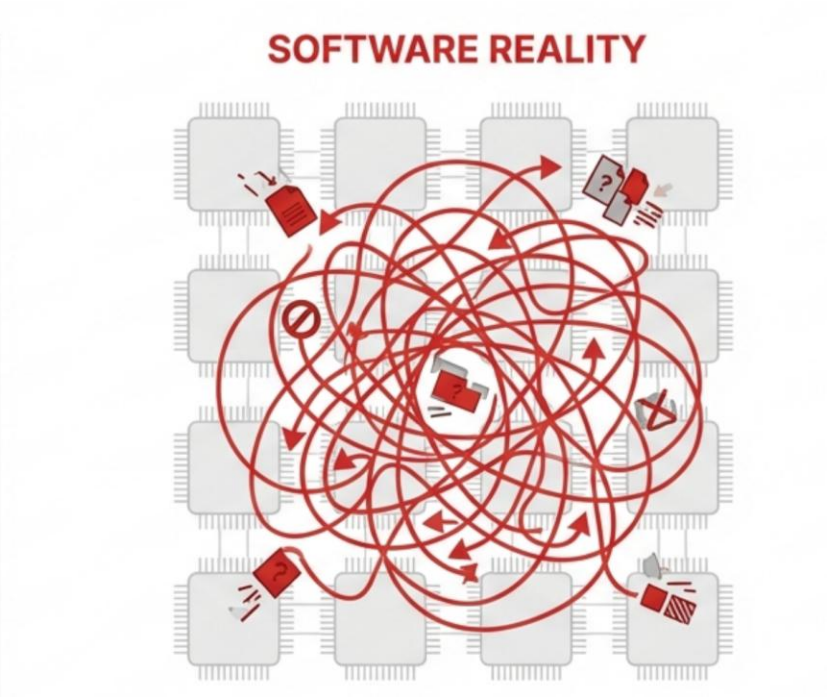
Inter-layer Scheduling



Introduction

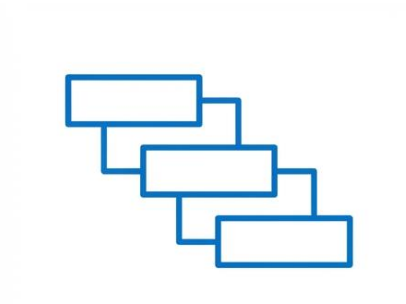
The Hardware Potential vs. The Software Bottleneck





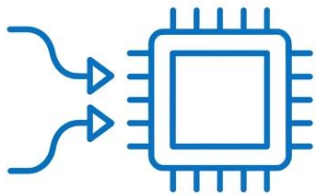
Introduction

4 key design factors for tiled architecture



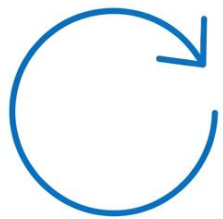
Execution Scheme (E)

Maps layers to cores temporally or spatially (Sequential, Pipeline, Parallel).



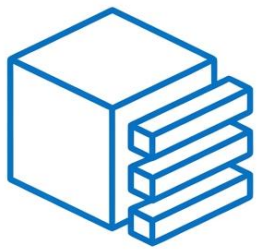
Fusion Strategy (F)

Fuses layers and keeps data on-chip to minimize DRAM access.



Recomputation (R)

Discards and recalculates intermediates to save memory (Vital for training).



Batch Splitting (B)

Splits into sub-batches for memory and pipeline efficiency.

Introduction

Comparison of various inter-layer schedulers

Inter-layer Scheduler	Execution Scheme	Fusion Strategy	Batch Splitting	Recomputation	Training Mode
SET / TileFlow	✓	✓	✓	✗	✗
Checkmate	✗	✗	Batch-level only	✓	✓
Tangram	✗	✓	✓	✗	✗
MBS	✗	✓	✓	✗	✓

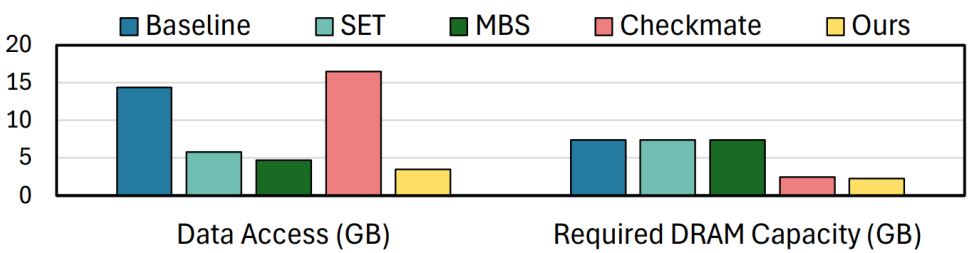
Challenge #1:
Incomplete Exploration of Design Factors
Lacks joint optimization of the 4 factors; specifically omits recomputation for training.

Challenge #2:
Constrained Scheduling Flexibility
Fixed sub-batch order; supports only linear structures.

Challenge #3:
Incomplete and Inefficient Search
Slow convergence and low solution quality due to reliance on heuristics.

Introduction

Training Comparison: Prior Arts vs. Crane



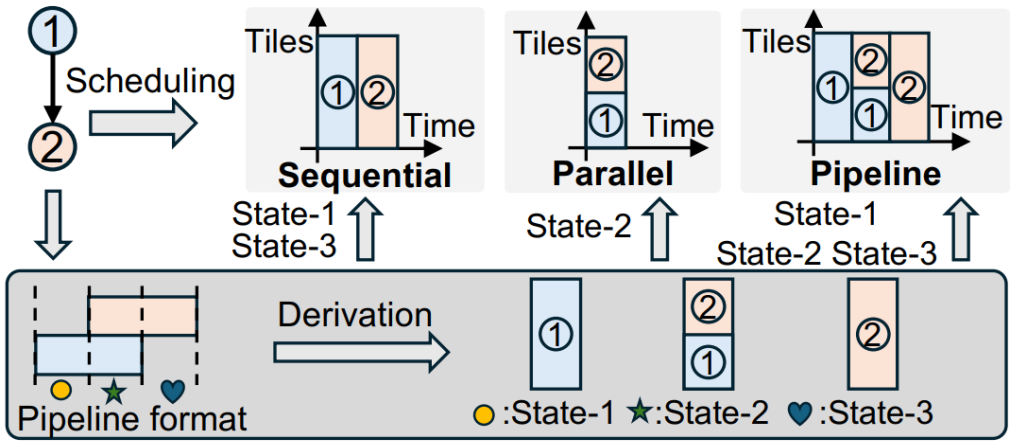
Memory Requirements in ResNet-50 Training (Batch=64)

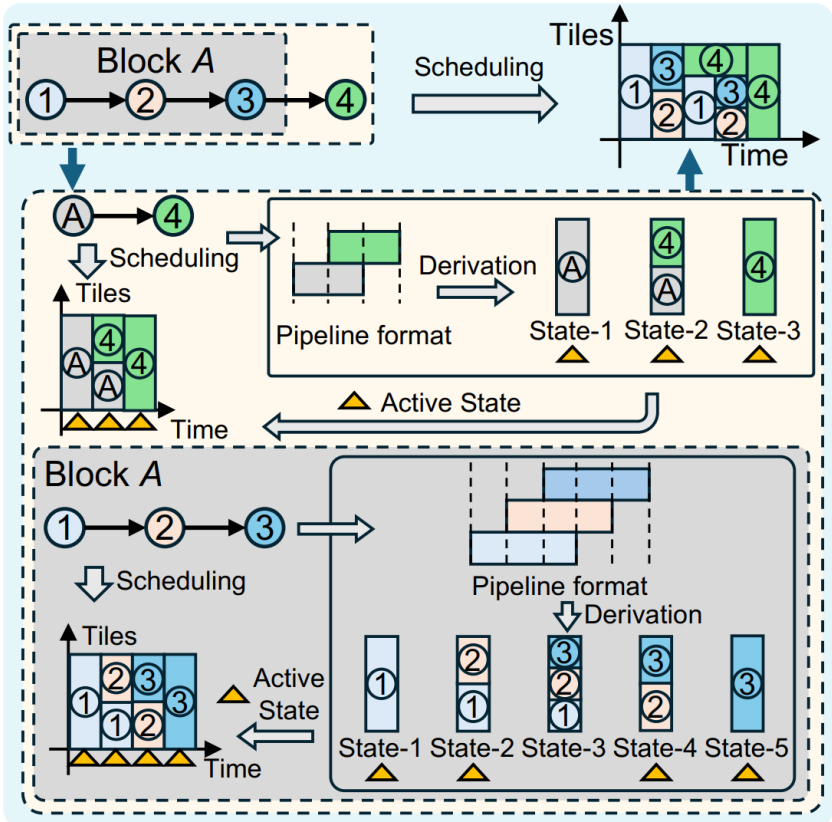
Framework	Data Access (Off-chip Traffic)	Required DRAM Capacity
SET / MBS	**Low** (Good)	**High** (Bad)
Checkmate	**High** (Bad)	**Low** (Good)
Crane (Ours)	**Low** (Best)	**Low** (Best)

Contribution

Contribution

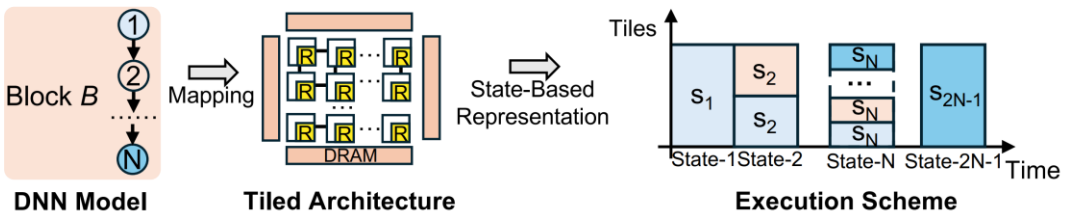
Hierarchical block representation and pipeline-derived states





Contribution

Scheduling Table (ScT): Fine-grained control of execution flow



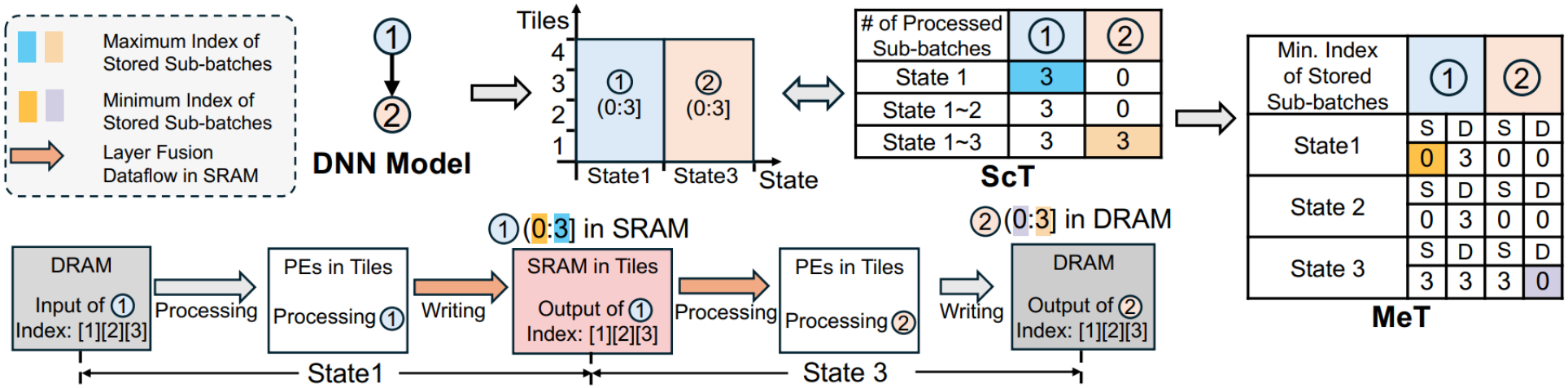
# of Processed Sub-batches	①	②	…	Ⓝ
State 1	s_1	0	…	0
State 1~2	$s_1 + s_2$	s_2	…	0
·····				
State 1~N	$s_1 + \cdots + s_N = BS/BS_{sub}$	$s_2 + \cdots + s_N$	…	s_N
·····				
State 1~2N-1	BS/BS_{sub}	BS/BS_{sub}	…	BS/BS_{sub}

ScT

- **Definition:**
  Records cumulative sub-batch counts processed by sub-blocks per state.
- **State-based Control:**
  Tracks execution progress over time.
- **Linear Constraints:**
  Table entries are defined as linear constraints, facilitating MILP optimization.

Contribution

Interaction between Memory Table (MeT) and Scheduling Table (ScT)



Contribution

Structured Optimization via MILP

ScT (Scheduling Table)

Pipeline States	Sub-blocks		
	Sub-block 1	Sub-block 2	Sub-block 3
State 1	3	3	3
State 2	3	3	3
State 3	3	3	3

MeT (Memory Table)

Pipeline States	Sub-blocks		
	Sub-block 1	Sub-block 2	Sub-block 3
State 1	[0:3]	[0:3]	[0:3]
State 2	[0:3]	[0:3]	[0:3]
State 3	[0:3]	[0:3]	[0:3]

Mixed-Integer Linear Programing (MILP)

Optimal Schedule

Feature	Heuristic Methods (Baselines)	MILP Method (Crane)
Algorithm	Sampling-based (Random)	**Structured Optimization (Mathematical)**
Search Space	Incomplete	**Exhaustive**
Convergence	Slow	**Fast**

Contribution

Recomputation example

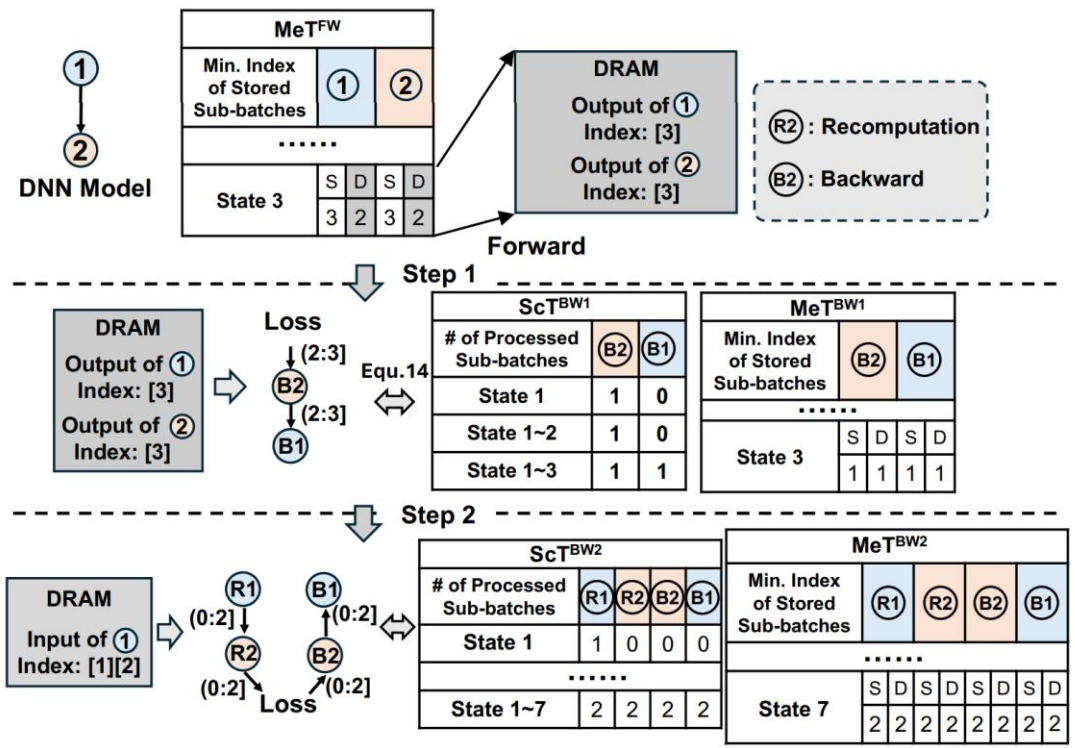
Step 1: Backward Pass-only
Backward propagation
using DRAM-cached activations

↓

Step 2: Recomputation
Forward re-execution
for evicted data recovery

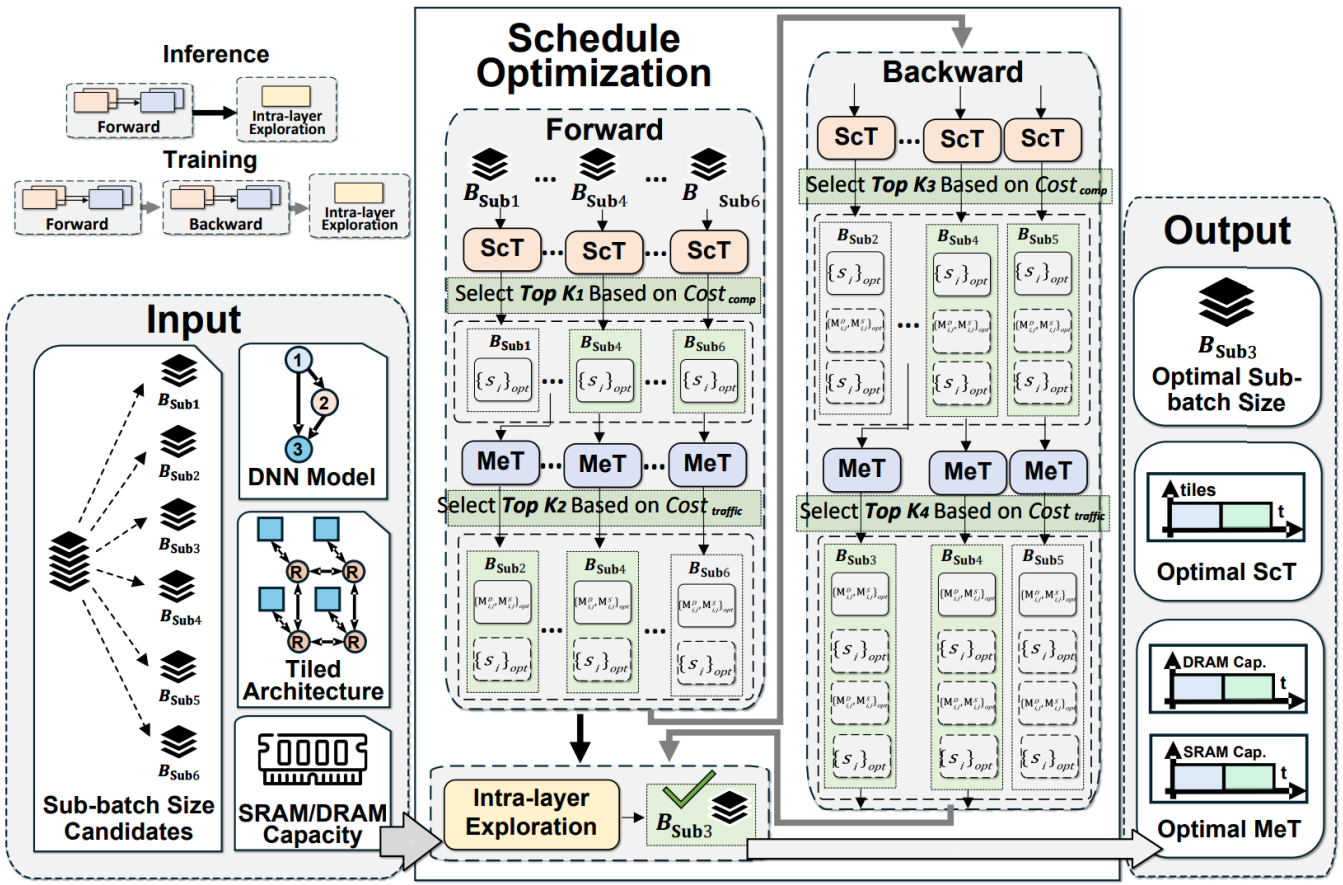
↓

Step 2-1: Backward Propagation
Backward propagation
using recomputed data



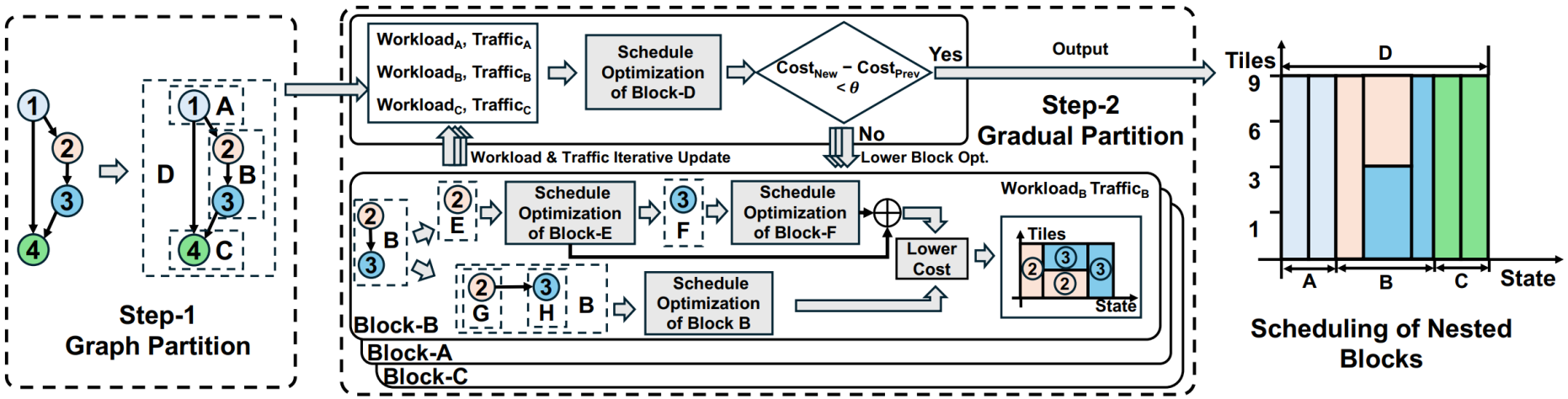
Contribution

Overall intra-block exploration and optimization process



Contribution

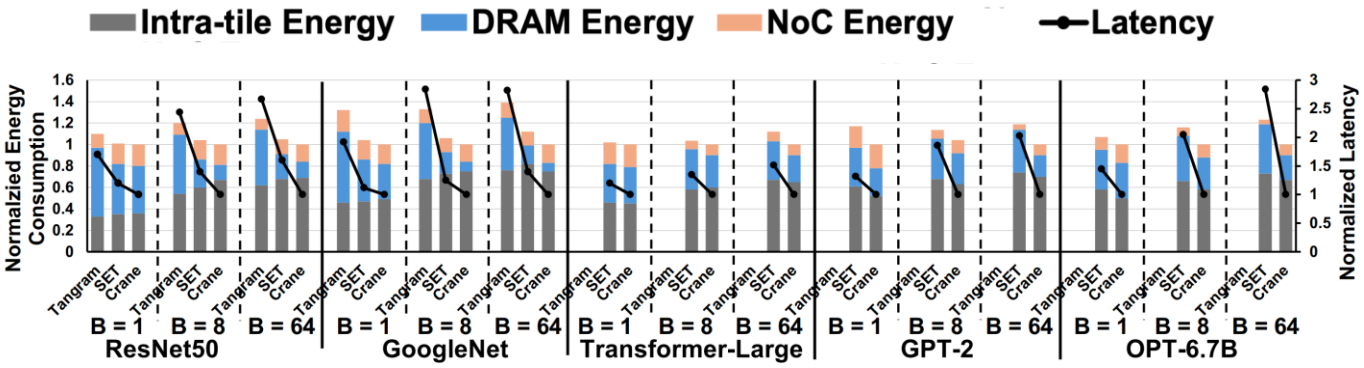
Hierarchical structure optimization



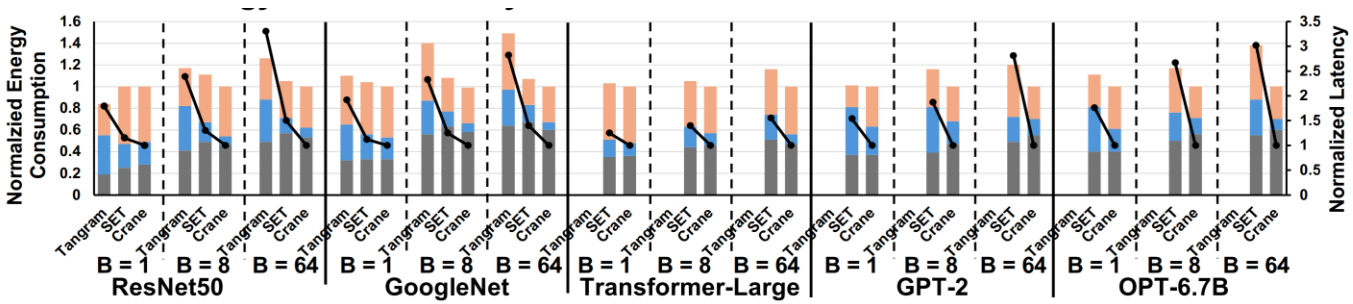
Evaluation

Evaluation

Inference Performance: General



(a) Edge-Side Architecture

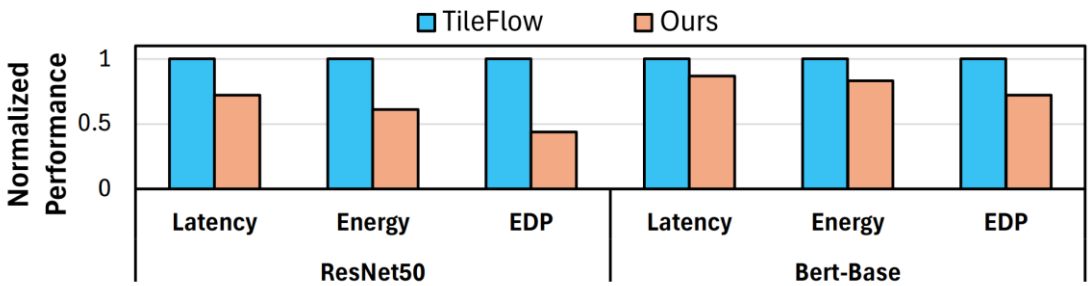


(b) Cloud-Side Architecture

Baseline	Metric	Reduction Range (Min ~ Max)
Tangram	Latency	**1.70× ~ 3.30×**
	EDP	**1.87× ~ 4.20×**
SET	Latency	**1.12× ~ 3.02×**
	EDP	**1.13× ~ 4.17×**

Evaluation

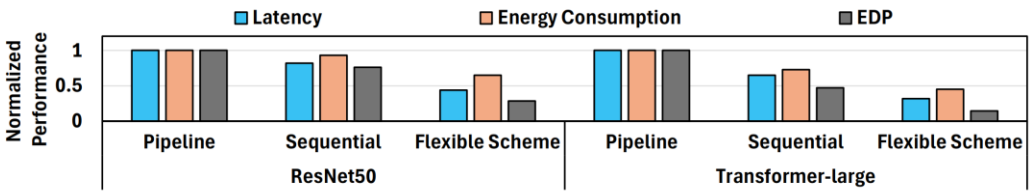
Inference Performance: Specialized



Model	Latency Reduction	EDP Reduction
ResNet-50	**28%**	**56%**
BERT-Base	**13%**	**28%**

Evaluation

Analysis of inference performance improvement



Metric	Average Reduction (Crane vs. Fixed Schemes)
Latency	**2.6× Lower**
Energy	**1.8× Lower**
EDP	**4.7× Lower**

Impact of Execution Scheme Exploration on Performance

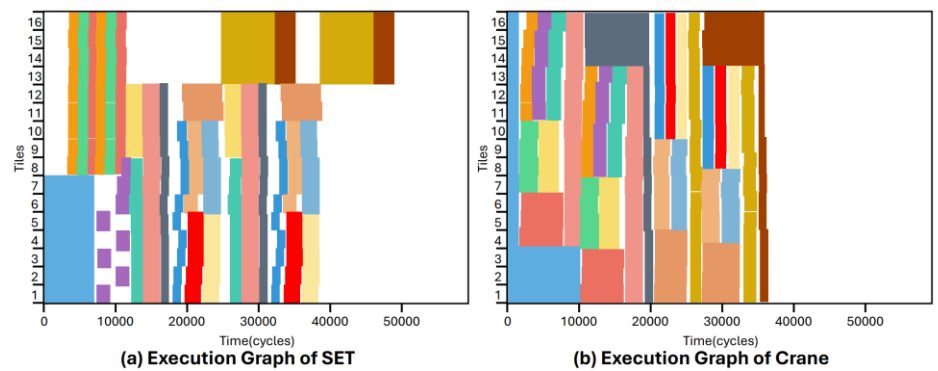
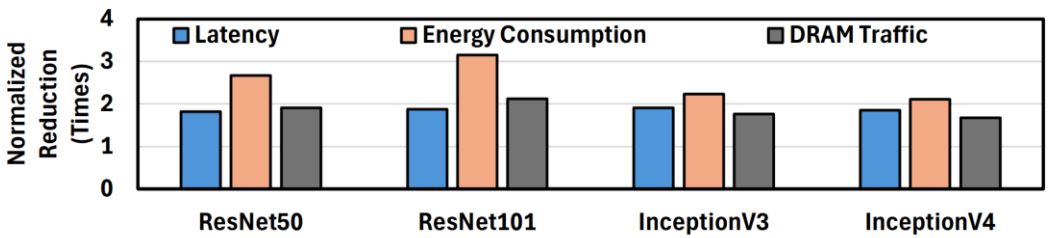


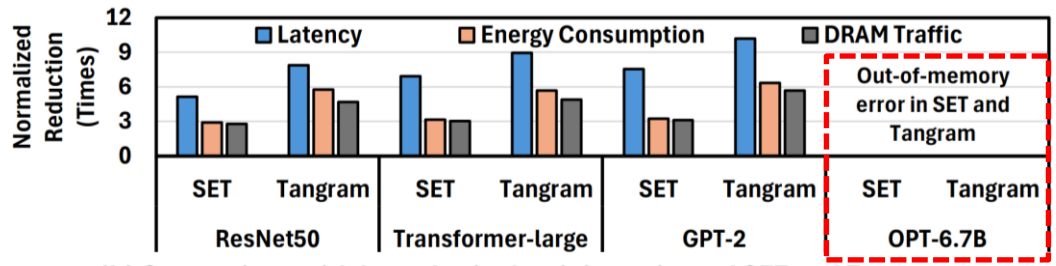
Figure 15: Tile-Time schedule for Inception inference (Crane vs SET).

Evaluation

Training Performance: Overcoming memory walls for large models



(a) Comparison with MBS



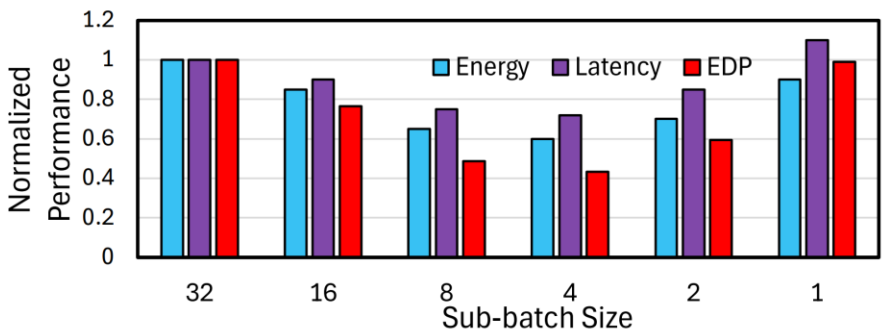
(b) Comparison with hypothetical training-oriented SET and Tangram

Metric	Improvement Range (Reduction)	Average Reduction
EDP	3.62× – 5.36×	4.18×
DRAM Traffic	1.42× – 2.22×	1.67×

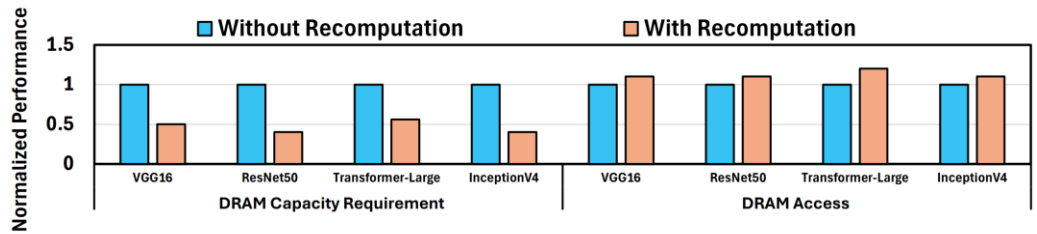
Metric	Baseline	Improvement Range (Reduction)
EDP	**vs. SET**	11.01× – 21.01×
	vs. Tangram	45.43× – 64.73×
DRAM Traffic	**vs. SET**	2.76× – 2.97×
	vs. Tangram	4.81× – 5.72×

Evaluation

Analysis of training performance improvement



Impact of Batch Splitting on ResNet-50 Training Performance



Metric	Configuration	Normalized Value (Avg.)
DRAM Capacity Requirement	**Without Recomputation**	**1.00**
	With Recomputation	**~0.45**
DRAM Data Access	**Without Recomputation**	**1.00**
	With Recomputation	**1.125**

Conclusion

Conclusion

Summary: The Crane Advantage

- **Unified Representation:** The Table-Format (ScT/MeT) successfully captures all design factors: Execution, Fusion, Recomputation, and Batch Splitting.
- **Structured Optimization:** Replaced slow heuristics with fast, exact MILP solvers.
- **SOTA Performance:** Unifies Inference & Training support.
  Reduces EDP by up to 21x. Enables massive models (OPT-6.7B) on tiled architectures.

Crane bridges the gap between complex DNN workloads and scalable Tiled Architectures.